Supervised assets

Issuer-freezable assets on Sequentia: what consensus can enforce, what it cannot, and who covers the difference. 2026-08-12, for Andreas and Alberto.

Proposal: add a declared asset class, *supervised*, whose issuer can freeze holders by consensus rule, and pair it with freeze-awareness in the Lightning and DEX implementations. Freezability is committed into the asset id at issuance, so it is permanent, visible before acquisition, and cannot be added to an existing asset.

The split is not a compromise, it is a boundary. Consensus can freeze what it can see and attribute: ordinary on-chain holdings, and resting covenant orders. It cannot freeze a Lightning balance, and no version of this rule ever will, because a channel's value lives in a shared output and in off-chain state that consensus cannot attribute to either party. That half must be enforced by the Lightning implementation as policy, which is exactly the division Andreas described.

The bar is parity with Ethereum, not improvement on it. That is what makes the Lightning boundary acceptable rather than a shortfall: whenever USDC sits inside a shared-state contract on Ethereum, blacklisting the holder's own address does not reach it either, because the contract holds the tokens while the holder's claim on them moves separately, without USDC ever being transferred. Circle's only lever there is to blacklist the *contract*, freezing everyone inside it, which it did in February 2026 (5.1). A Lightning channel is that same shape. So a supervised asset reaches a holder's own balance exactly as an EVM blacklist does, and stops exactly where an EVM blacklist stops.

1. The requirement, and the one part of it that cannot be met

The requirement is that Circle can freeze USDC balances, while USDC otherwise behaves normally on the DEX and on Lightning; that a blacklisted holder's channels stop working and their resting orders never complete; and that breakage is clean, harming no one else.

Everything in that is achievable except one thing, and it is worth stating before the design so the design is read correctly.

A consensus rule cannot freeze a Lightning balance. A channel's funds sit in a single 2-of-2 output that holds *both* parties' claims, and the split between them exists only in off-chain state. Consensus sees one output and cannot tell whose money is inside. That leaves exactly two options and no third:

- **Do not freeze the funding output.** Then a frozen holder with a channel is not frozen at all: they can move their whole balance off-chain, where no consensus spend occurs, or co-sign a close paying everything to a fresh script.

- **Freeze the funding output.** Then the innocent peer's entire balance is trapped permanently, with no unilateral escape, because every exit from a channel spends that one output.

Related, and equally unfixable at the consensus layer: an issuer cannot tell from chain data whether freezing a holder's `to_local` output is legitimate or is destroying the peer's penalty claim against a cheat, because the two are byte-identical and the difference is private off-chain data. And because Lightning's remedies are races against absolute deadlines, a *temporary* freeze on a shared script does not delay anyone; it silently hands the disputed value to the frozen party.

So Lightning is not a gap in this proposal to be closed later. It is the boundary of what consensus can do, and the proposal is built around it rather than pretending otherwise.

2. What consensus does

2.1 A supervised asset is declared at birth

The issuance commits an **issuer freeze key** into the asset id. Freezability therefore cannot be added to an existing asset or removed from a supervised one, and a freeze record is valid only if signed by that key. Ordinary assets are untouched by every rule below.

2.2 Enforcement is on spending, and only for single-owner outputs

Consensus rejects a transaction that *spends* a supervised-asset output when two things hold: the output's script is frozen, and the spend reveals a **single-owner** script (a key-path or single-signature spend). Creating outputs that pay a frozen script stays legal.

Both halves matter, and each removes a specific way the naive rule destroyed third-party funds:

- **Spend-only** means an innocent party can always be *paid*. Nobody's incoming funds can be blocked because their counterparty is frozen.
- **Single-owner-only** means a freeze can never bite on a script that holds someone else's claim. A 2-of-2, an HTLC, a covenant with a refund branch: the spend reveals the script, consensus sees it is not single-owner, and the freeze does not apply. This is decidable at spend time even though it is undecidable at freeze time, because the witness reveals the script that the address only committed to as a hash.

The cost of that second rule is explicit: a holder who moves supervised funds into a shared script before being frozen has escaped the consensus rule. That is the same escape a holder gets on Ethereum by moving USDC into a pooled contract, where their claim becomes a separate token that changes hands without USDC ever being transferred. It is why section 3 exists.

2.3 Resting DEX orders are covered, and cleanly

A covenant order is the good case, and it is worth separating from Lightning because the difference is instructive: a resting covenant output holds **only the maker's own asset**, so freezing it traps nobody

else. The order's script is publicly derivable from the order parameters, so an issuer can freeze it. A taker who tries to fill afterwards has their transaction rejected and loses nothing but the attempt.

For this to be clean rather than merely safe, the DEX must reap frozen orders from the book, which needs a signal it does not have today (4.2).

2.4 Supervised assets are explicit

Consensus cannot see which asset a blinded output holds, so a supervised asset must never be blinded. This is not free, and the cost is in a place worth naming precisely: **the node wallet currently blinds change automatically** whenever a transaction has two or more change outputs, which under any-asset fees is the ordinary case for sending an issued asset. Shipping the rule without fixing that would break ordinary sends. Both the node wallet and the DEX's own wallet daemon need the change to keep supervised change explicit.

3. What the Lightning and DEX implementations must do

This is the other half of the design, not an afterthought, and it is where a blacklisted holder's channels actually stop working.

- **Refuse to open** a supervised-asset channel with a blacklisted peer.
- **Refuse to forward or route** supervised-asset payments for a blacklisted peer, which is what makes the channel stop working in practice.
- **Close on blacklist.** On learning a peer is frozen, initiate a close. The commitment pays the frozen party's own script, so the balance lands on-chain where consensus can hold it, and the innocent peer is paid out normally and can spend immediately.
- **Never freeze the funding script**, as an operational rule for the issuer. Consensus already declines to apply a freeze to a shared script under 2.2, so this is defence in depth rather than the only line.
- **DEX: reap frozen orders** from the book so takers never attempt a fill that consensus will reject.

The honest consequence: between a holder being blacklisted and their channels closing, they can still move value inside Lightning. That window is a policy matter for the Lightning implementation, not something consensus can shorten.

4. What must ship with the rule

4.1 Mempool eviction

A spend that was valid on entry becomes invalid the moment a freeze confirms, and nothing removes it today: the mempool drops only transactions included in a block or directly conflicting, and a freeze record conflicts with nothing. The stale transaction is re-selected into every block template, template construction fails, and producers skip slots — a chain halt, not an inconvenience. The node also

asserts this cannot happen: `CTxMemPool::check` re-runs `CheckTxInputs` inside a bare `assert` commented "should always pass" (`src/txmempool.cpp:920`), so a freeze would abort nodes. Eviction on freeze, re-admission on unfreeze, and a reconsidered assert are part of the feature, not follow-up work.

4.2 A freeze signal tools can read

An RPC answering "is this asset supervised" and "is this script frozen", plus a distinct machine-readable rejection reason. Without it the DEX cannot reap orders, wallets cannot explain a failure, and the Lightning implementation cannot act on a blacklist at all. Section 3 is unimplementable without this.

4.3 Wallet change handling

Per 2.4: keep supervised-asset change explicit in the node wallet and in the DEX's wallet daemon. This is the item most likely to be discovered late and to look like an unrelated bug.

4.4 Activation and cutover

Height-gated above the tip at release, with the same all-nodes-at-once discipline as the Simplicity activation and the 60-second-block fork, because an un-upgraded node diverges at the gate.

5. Precedent and limits

5.1 The case study: Circle and Zama, February 2026

The dilemma in section 1 is not a quirk of Lightning or of the UTXO model. It is what happens whenever a shared construct holds several parties' value, and Ethereum met it in the most direct way possible earlier this year.

Zama's confidential USDC wrapper is a contract that pools USDC on behalf of many holders. A US federal court order, arising from litigation against a single individual who was not the protocol, required Circle to act. Circle's only available lever was the address that actually held the tokens, so it blacklisted **the contract**. Roughly **$12.6 million** of USDC was frozen inside, belonging to holders who were not parties to the case and had done nothing. Liquidity providers could not withdraw and the pool was effectively dead, since a blacklisted address can neither send nor receive.

Read that against section 1 and the shapes are identical. The target's value was inside a shared construct; blacklisting the target's own address would not have reached it; blacklisting the construct reached everybody. Circle faced exactly the two options this proposal faces with a channel funding output, and took the second one.

Why this matters for the design rather than merely as background. The single-owner-only rule in 2.2 makes a Zama-style outcome *impossible on Sequentia by construction*. An issuer here cannot freeze a shared script even if it wants to, because consensus declines to apply the freeze once the spend reveals a script that is not single-owner.

That is the one place this design deliberately gives an issuer *less* than Ethereum does, and it should be raised with Circle rather than glossed. The power being declined is the one that produced $12.6 million of collateral damage and a public backlash, and declining it structurally means nobody can ever be ordered to inflict that here. If Circle regards a contract-level freeze as a requirement rather than a last resort, that is a conversation to have before any of this is built, not after.

It also disposes of a weaker comparison I made earlier. Balances at an exchange are not a good analogy, because an exchange is a legal person a court can order to freeze an account. A pooled contract is not, and neither is a Lightning channel. Shared constructs, not custodians, are the real limit of what any blacklist reaches.

5.2 Limits, stated so nobody discovers them later

- **It freezes coins at a script, not a person.** A holder spread across twenty addresses must be frozen twenty times, and the freezing transaction is public before it confirms, so an attentive target can move first. Against a custodian, an exchange deposit address or a wallet identified by analysis it works; against an evading self-custody holder it is a race.
- **Shared scripts are outside its reach**, by design, because the alternative is trapping innocent counterparties.
- **Off-chain balances are outside its reach**, permanently. Lightning, and any future venue holding pooled balances, must enforce at their own layer.
- **It is not retroactive**, and must not be: a rule that rejects spends cannot apply to history produced before it existed.

6. What this means for USDC: issue it supervised

`USDC.e` has zero supply and zero deposits. Supervision can only be conferred at issuance, so the choice is live now and closes permanently the moment real USDC is bridged.

Recommendation: re-issue `USDC.e` as a supervised asset, now, without waiting to be asked. Circle publishes the Bridged USDC Standard precisely so that a third party can conform to it unilaterally and present a finished, conforming asset; the standard's central instruction is that the upgrade path must be built in at deployment because it cannot be applied retroactively. Supervision is exactly such a property. An asset issued without it can never gain it, so deferring the decision is not neutral, it is choosing the unfreezable option permanently and hoping that never matters.

The case for choosing it is short. Native USDC is freezable on every chain Circle issues on, without exception. An asset that cannot be frozen is not a candidate for in-place adoption by an issuer whose token has that property everywhere else, whatever else it gets right. Parity is the bar, and supervision is what parity requires.

The case against is thinner than it first looks, because the design no longer trades composability for control. A supervised asset still works in Lightning channels, still rests on the covenant order book, still settles same-chain swaps, and still pays its own fees. The single cost is confidentiality: supervised

assets are always explicit. This standard already requires `USDC.e`'s supply events to be explicit so that backing is externally auditable, and a dollar whose issuer must answer for its flows is a poor candidate for blinding in any case. So the practical loss is close to zero.

Property	Ordinary USDC.e (today)	Supervised USDC.e (proposed)
Issuer freeze	Never, for anyone, permanently	Holder's own on-chain balance and resting orders
Lightning	Works	Works
Covenant order book	Works	Works
Confidentiality	Available, opt-in	Never
Adoptable in place by Circle	Only if Circle accepts no freeze	Yes, at parity with its other chains

The sequencing matters and is cheap. Re-issuing costs one transaction while supply is zero, and the registry entry and bridge configuration are a few lines. The alternative is discovering in a due-diligence conversation a year from now that the asset cannot be adopted and cannot be fixed.

But this is a consensus change, so when the asset can be issued depends on an answer we have not settled. The rule needs an activation height (4.4). Whether `USDC.e` can be re-issued supervised *before* that height turns on how the freeze key is committed, which 7.2 asks Alberto and which is therefore a scheduling decision rather than a matter of taste.

- **A distinct derivation constant** changes how the asset id is computed, and consensus computes that id rather than reading it from the transaction (`CalculateAsset`, `src/confidential_validation.cpp:219`). A node without the rule would derive a different id for the same issuance, so the issuance is a chain split until every node has the new binary. Under this encoding the asset cannot be issued until after activation, and **USDC deposits must stay closed until then**, since any USDC bridged in the meantime lands in an asset that can never be supervised.

- **A declaration output in the issuance transaction** leaves derivation untouched, so every node computes the same asset id and there is no split. The asset can be issued supervised today; the declaration sits inert, and enforcement simply begins at the activation height. This decouples the asset decision from the fork schedule entirely.

The second is strictly better for sequencing and is what I would choose unless it fails review on other grounds. It is the difference between shipping USDC on our own timetable and holding the bridge closed until a consensus fork lands.

7. For Alberto

1. **Single-owner detection at spend time.** The rule turns on classifying a revealed script as single-owner. P2WPKH and P2TR key-path are clear. Is a bare P2PK or a 1-of-1 multisig worth including, and where should the classifier live so that mempool and block validation cannot disagree?
2. **Where the freeze key is committed, which is really a scheduling question.** A distinct derivation constant makes the class visible from the id alone but cannot be issued before the fork activates without splitting the chain; a declaration output in the issuance transaction leaves derivation untouched and can therefore be issued today, inert until activation. See the box in section 6: this decides whether USDC deposits can open before the fork or must wait for it.
3. **Mempool eviction design.** This is the piece most likely to go wrong, and it touches code we both rely on. I would want your review of the eviction and re-admission paths specifically.
4. **Whether supervised assets may pay fees**, given a frozen holder's fee output interacts with producer fee handling.

8. Provenance

This design is what survived ten independent analyses run against the real trees, whose blocker-rated findings were then attacked by separate reviewers. Two earlier shapes were discarded by that process: freezing arbitrary scripts of any type, which strands innocent counterparties and converts Lightning's timelock races into thefts; and restricting supervised assets to single-owner outputs entirely, which would have banned them from the DEX and Lightning outright and cost more than it bought.

Claims about the mempool assert and about whole-transaction rejection were re-verified by hand. One reported finding was discarded as an artifact of the shared `~/Sequentia` checkout, which sits on `pos-exprace` and still shows Simplicity inactive while master has it active and testnet proved it in block 89860; that checkout should be brought up to date, since other sessions read it as ground truth.

Prepared by Saba. No consensus code has been written; a sketch produced during analysis was reverted unbuilt.